

AULA 07

Transações, ACID e 2PC

Banco de Dados II

Prof. Alysson

26/09/2026

Recapitulando a Aula 06

- **Estruturas de controle:** IF , CASE , LOOP , WHILE , FOR
- **Exceções:** BEGIN ... EXCEPTION WHEN ... END , RAISE , SQLSTATE
- **Cursores:** percorrer um resultado linha a linha
- **Gatilhos:** a regra que o próprio servidor dispara — NEW , OLD , TG_OP

A Aula 06 ensinou o banco a **reagir**. Hoje ele aprende a **prometer**:
ou faz tudo, ou não faz nada.

Uma pista que já estava na Aula 06

```
BEGIN
    UPDATE produto SET estoque = estoque - p_qtd WHERE id = p_id;
EXCEPTION
    WHEN check_violation THEN
        RAISE NOTICE 'Estoque insuficiente';
END;
```

- Quando o erro acontece, o `UPDATE` do bloco é **desfeito** — só ele
- Para isso o PostgreSQL cria um **ponto de salvamento implícito** no início do bloco com `EXCEPTION`
- Hoje esse ponto ganha nome: **SAVEPOINT**

Onde estamos no semestre

Aulas	Bloco	Pergunta que responde
01–04	Fundamentos e arquitetura	Como o banco guarda e organiza os dados?
05–06	Programação no servidor	Como pôr regras dentro do banco?
07–08	Confiabilidade	Como o banco não perde nem estraga os dados?
09	A1 — 10/10	Prova com as Aulas 01 a 06

- **Hoje:** transações, ACID e transações distribuídas (2PC)
- **Aula 08:** concorrência, MVCC, bloqueios e níveis de isolamento

Pergunta de partida

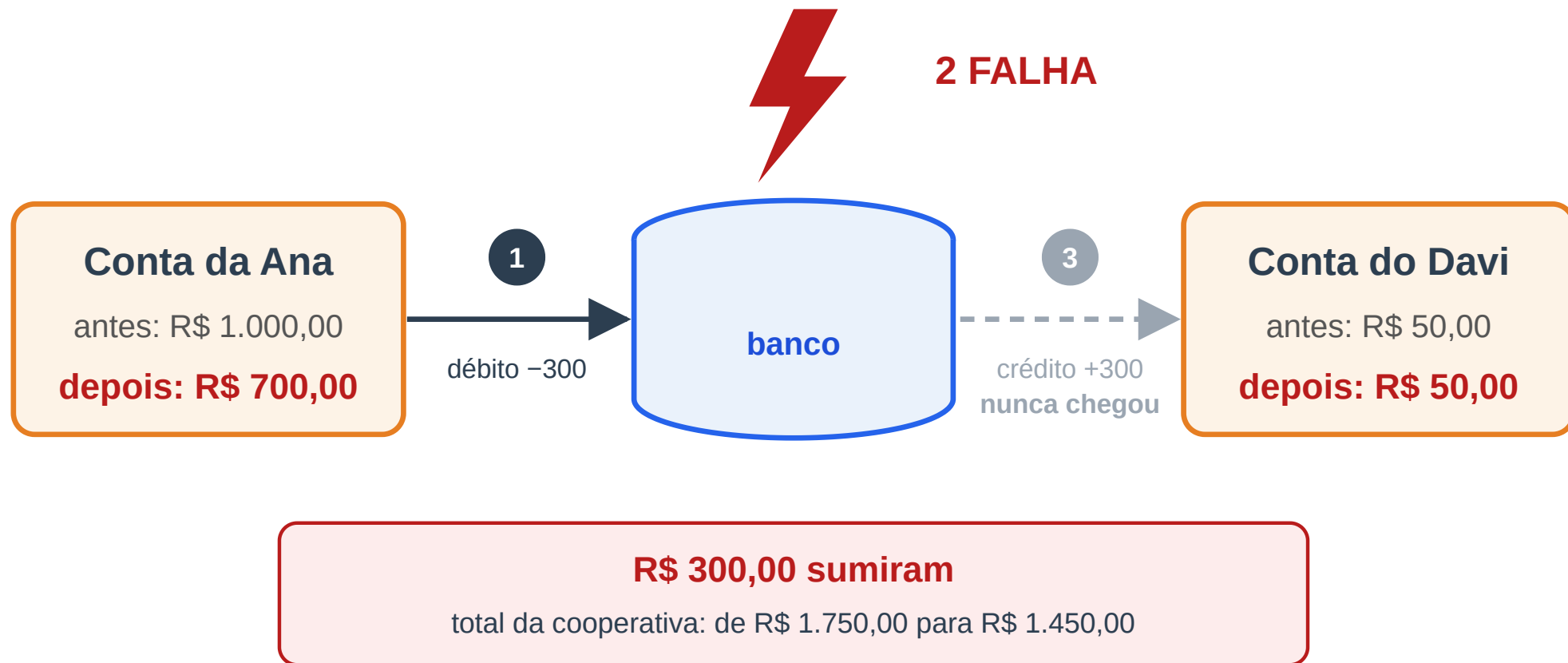
Ana transfere R\$ 300,00 para Davi. São dois passos:

1. tirar R\$ 300,00 da conta da Ana
2. pôr R\$ 300,00 na conta do Davi

E se a energia acabar entre o passo 1 e o passo 2?

- A Ana perdeu R\$ 300,00
- O Davi não recebeu nada
- O dinheiro **sumiu** — e o banco perdeu a confiança do cliente

A transferência que some



Hoje vamos aprender

- O que é uma **transação** e quais problemas ela resolve
- As quatro propriedades **ACID** — e quem garante cada uma no PostgreSQL
- Os comandos de controle: **BEGIN** , **COMMIT** , **ROLLBACK** e **SAVEPOINT**
- O que acontece quando **um erro** aparece no meio da transação
- Por que um **COMMIT** não basta quando os dados estão em **dois servidores**
- O protocolo de **confirmação em duas fases (2PC)**

Como esta aula funciona

- **No encontro (1h30):** conceitos, código lido junto e duas demonstrações curtas no mesmo ambiente que você vai usar em casa
- **Não há atividade em sala:** nada a entregar hoje
- **Em casa:** o laboratório no iximiuz Labs e as dez questões no Classroom

O laboratório tem **sete atividades independentes**, de **no máximo 20 minutos** cada. A conta gratuita do iximiuz dá cerca de **1 hora por dia** e apaga o ambiente ao sair — então cada atividade **começa do zero** e não depende de nenhuma outra.

Parte 1

Transação e ACID

O que é uma transação

- Uma **sequência de operações** tratada como **uma coisa só**
- Ou **todas** as operações acontecem...
- ...ou **nenhuma** acontece
- Leva o banco de um **estado válido** para outro **estado válido**

Transação = unidade lógica de trabalho, **indivisível**.

A transferência é **uma** transação com **dois** UPDATE .

O cenário da aula: a Cooperativa Cerrado

```
CREATE TABLE conta (  
    id          integer          PRIMARY KEY,  
    titular     varchar(40)      NOT NULL,  
    cidade      varchar(30)      NOT NULL,  
    saldo       numeric(10,2)    NOT NULL CHECK (saldo >= 0)  
);
```

- `CHECK (saldo >= 0)` : **nenhuma conta pode ficar negativa**
- É a mesma tabela do laboratório, no banco `banco`

As contas da Cooperativa Cerrado

id	titular	cidade	saldo
1	Ana	Palmas	1000.00
2	Bruno	Palmas	500.00
3	Carla	Gurupi	200.00
4	Davi	Gurupi	50.00

Total da cooperativa: R\$ 1.750,00 — nenhuma transferência deve mudar isso.

Os dois inimigos de uma transação

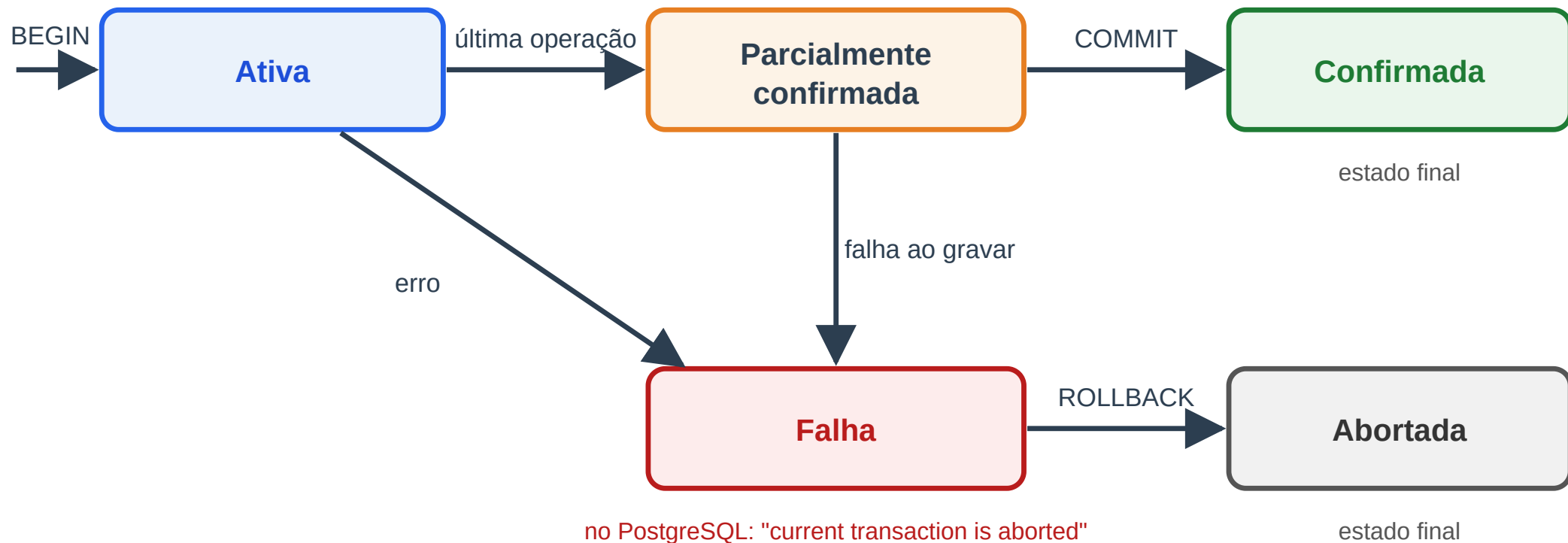
Inimigo	Exemplo	O que pode dar errado
Falha	queda de energia, disco cheio, processo derrubado	a operação fica pela metade
Concorrência	dois caixas mexendo na mesma conta ao mesmo tempo	uma alteração apaga a outra

- Contra a falha: **desfazer** o que ficou pela metade e **refazer** o que foi confirmado
- Contra a concorrência: **isolar** uma transação da outra

Estados de uma transação

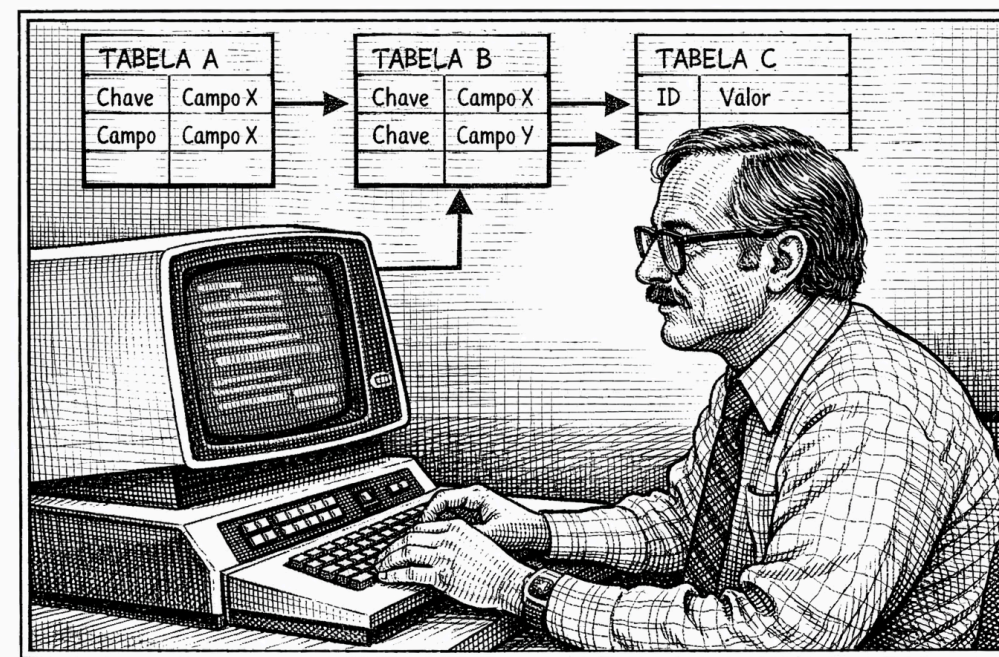
Estado	O que significa
Ativa	as operações estão sendo executadas
Parcialmente confirmada	a última operação terminou; falta gravar
Confirmada (<i>committed</i>)	tudo gravado — não há volta
Falha	um erro interrompeu a execução normal
Abortada (<i>aborted</i>)	tudo foi desfeito; o banco voltou ao estado anterior

Estados de uma transação — diagrama



Um pouco de história: Jim Gray

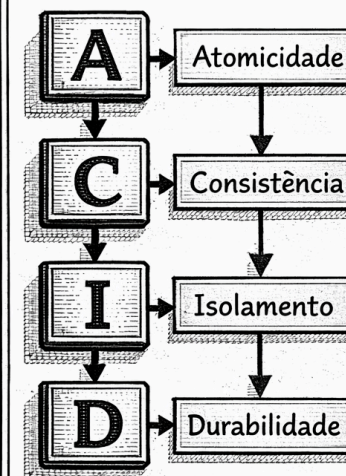
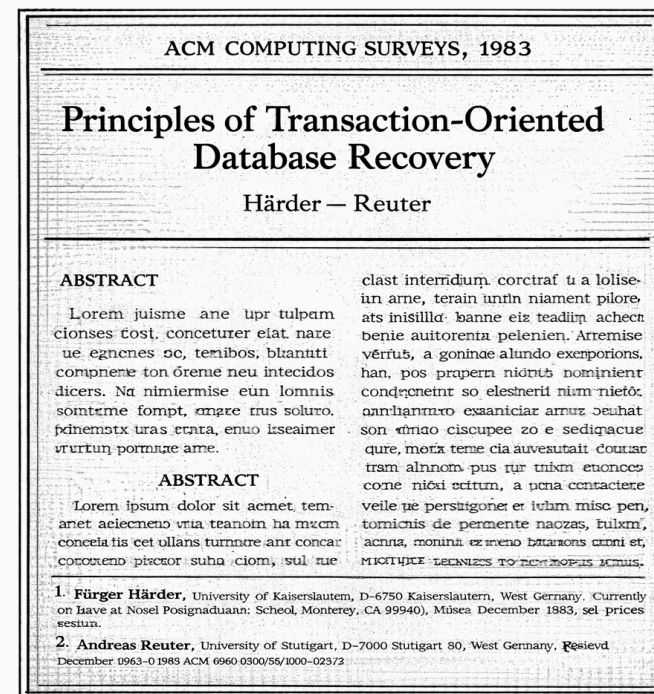
- IBM, projeto **System R** (anos 1970)
- Formalizou **transação** e **recuperação**
- Descreveu o **2PC** em 1978
- Prêmio Turing em 1998



Jim Gray — IBM System R, 1974

Um pouco de história: Härder e Reuter

- **Theo Härder** e **Andreas Reuter**, pesquisadores alemães
- Artigo de **1983** sobre recuperação de bancos de dados
- Criaram o acrônimo **ACID**
- Todo SGBD relacional moderno segue esse contrato



ACID — visão geral

Letra	Propriedade	A promessa em uma frase
A	Atomicidade	Tudo ou nada
C	Consistência	O banco sai de um estado válido para outro válido
I	Isolamento	Uma transação não atrapalha a outra
D	Durabilidade	O que foi confirmado , fica — mesmo com falha

ACID é o **contrato** entre o SGBD e quem escreve a aplicação.

A — Atomicidade

- A transação é **indivisível**: ou tudo, ou nada
- Se **qualquer** operação falhar, **todas** são desfeitas
- Na transferência: o débito na Ana e o crédito no Davi **andam juntos**

```
BEGIN;  
UPDATE conta SET saldo = saldo + 80 WHERE id = 3;  -- Carla: funciona  
UPDATE conta SET saldo = saldo - 80 WHERE id = 4;  -- Davi: falha (saldo 50)  
COMMIT;                                           -- responde ROLLBACK  
-- resultado: Carla continua com 200.00 – o crédito também foi desfeito
```

C — Consistência

- A transação respeita **todas as regras** do banco
- Regras do SGBD: `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `CHECK`, gatilhos
- Regras do negócio: "o total da cooperativa não muda numa transferência"

Quem garante	O quê
SGBD	as restrições declaradas e os gatilhos
Desenvolvedor	que a transação faça a coisa certa (debitar e creditar)

O `CHECK (saldo >= 0)` barra a transferência do Davi. Mas nenhuma restrição impede um programa de debitar sem creditar — isso é seu.

I — Isolamento

- Transações **ao mesmo tempo** não devem se atrapalhar
- Cada uma trabalha **como se estivesse sozinha** no banco
- No PostgreSQL, uma sessão **não vê** o que a outra ainda não confirmou
- Duas transações que querem **a mesma linha**: a segunda **espera**

O isolamento tem **níveis**, e cada nível tem um custo. Esse é o tema inteiro da **Aula 08** — hoje só vamos vê-lo funcionar.

D — Durabilidade

- Depois do `COMMIT`, a alteração **não se perde** — nem com queda de energia
- Quem garante é o **WAL** (*Write-Ahead Log*), visto na Aula 04:
 - a alteração vai **primeiro para o diário**, depois para os dados
 - o `COMMIT` só responde com o diário **gravado em disco**
 - depois de uma queda, o servidor **refaz** o diário (*redo*)

No laboratório: `podman kill` e, no log, *automatic recovery in progress*.

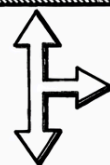
ACID — as quatro garantias

Operações são atômicas.

A
Atomicidade

Estado final é consistente.

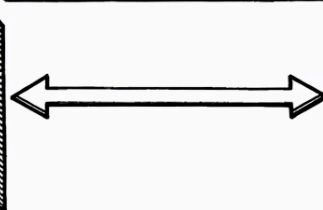
C
Consistência



TRANSAÇÃO



I
Isolamento



D
Durabilidade

Execução isolada das transações.

Dados sobrevivem a falhas.

Quem garante cada letra no PostgreSQL

Letra	Mecanismo no PostgreSQL	Onde você vê
A	desfazimento da transação inteira em caso de erro	<code>COMMIT</code> que responde <code>ROLLBACK</code>
C	restrições (<code>CHECK</code> , <code>FOREIGN KEY</code> ...) e gatilhos	<code>violates check constraint</code>
I	MVCC (versões das linhas) e bloqueios de linha	a outra sessão não vê / espera
D	WAL gravado em disco no <code>COMMIT</code>	<code>automatic recovery in progress</code>

Resumo da Parte 1

- **Transação:** várias operações que valem como **uma**
- **Inimigos:** a **falha** e a **concorrência**
- **ACID:** Atomicidade, Consistência, Isolamento e Durabilidade
- **História:** Jim Gray (System R, 2PC) e Härder e Reuter (o nome ACID, 1983)
- **No PostgreSQL:** restrições, MVCC, bloqueios e WAL

Parte 2

Controlando a transação no PostgreSQL

Os comandos de controle (TCL)

Comando	O que faz
BEGIN OU START TRANSACTION	abre uma transação
COMMIT	confirma tudo desde o BEGIN
ROLLBACK	desfaz tudo desde o BEGIN
SAVEPOINT nome	marca um ponto seguro dentro da transação
ROLLBACK TO SAVEPOINT nome	volta até o ponto seguro, sem encerrar a transação
RELEASE SAVEPOINT nome	descarta o marcador (não confirma nada)

TCL = *Transaction Control Language*.

Autocommit: o padrão do PostgreSQL

- **Sem** `BEGIN`, cada comando é uma transação **sozinha**, confirmada na hora
- Não há como desfazer depois

```
UPDATE conta SET saldo = saldo - 10 WHERE id = 1;  
ROLLBACK;
```

```
UPDATE 1  
WARNING:  there is no transaction in progress  
ROLLBACK
```

- A Ana perdeu R\$ 10,00 de verdade: o `ROLLBACK` chegou tarde
- O `psql` e o pgAdmin trabalham em **autocommit** por padrão

BEGIN e COMMIT — a transferência certa

```
BEGIN;  
UPDATE conta SET saldo = saldo - 100 WHERE id = 1;    -- débito em Ana  
UPDATE conta SET saldo = saldo + 100 WHERE id = 2;    -- crédito em Bruno  
COMMIT;  
  
SELECT id, titular, saldo FROM conta ORDER BY id;
```

id	titular	saldo
1	Ana	900.00
2	Bruno	600.00
3	Carla	200.00
4	Davi	50.00

ROLLBACK — desistir no meio

```
BEGIN;  
UPDATE conta SET saldo = saldo - 50 WHERE id = 2;  
UPDATE conta SET saldo = saldo + 50 WHERE id = 3;  
SELECT id, titular, saldo FROM conta ORDER BY id;  
-- dentro da transação: Bruno 550.00 e Carla 250.00  
  
ROLLBACK;  
SELECT id, titular, saldo FROM conta ORDER BY id;  
-- de volta: Bruno 600.00 e Carla 200.00
```

- As duas alterações são desfeitas **juntas**
- Quem estava fora da transação **nunca viu** os valores intermediários

Um erro no meio da transação

```
BEGIN;  
UPDATE conta SET saldo = saldo + 80 WHERE id = 3;    -- UPDATE 1  
UPDATE conta SET saldo = saldo - 80 WHERE id = 4;
```

```
ERROR:  new row for relation "conta" violates check constraint "conta_saldo_check"  
DETAIL:  Failing row contains (4, Davi, Gurupi, -30.00).
```

```
SELECT * FROM conta;
```

```
ERROR:  current transaction is aborted, commands ignored until end of transaction block
```

A transação está **abortada**: o servidor ignora tudo até ela terminar.

O COMMIT que vira ROLLBACK

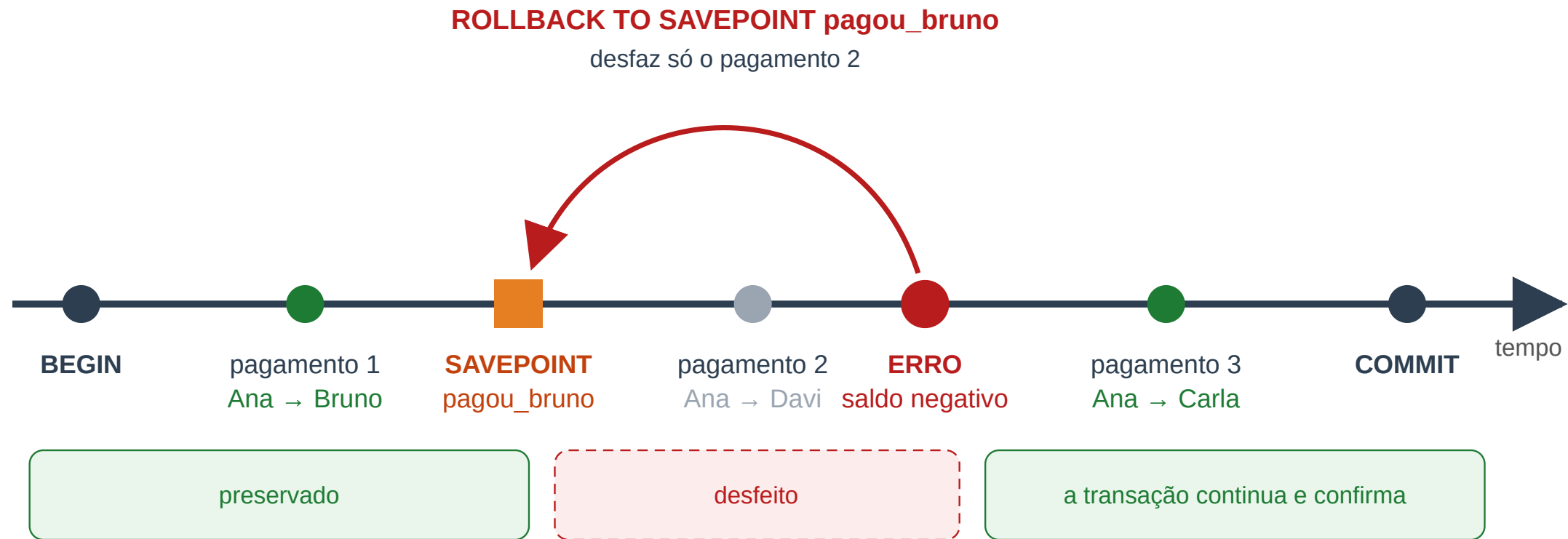
```
COMMIT;
```

```
ROLLBACK
```

- Você pediu COMMIT, e o servidor respondeu ROLLBACK
- Uma transação abortada não pode ser confirmada — nem em parte
- O crédito de R\$ 80,00 na Carla, que tinha funcionado, também foi desfeito

Atomicidade em ação. Na aplicação, confira sempre a resposta do COMMIT: ele pode ter falhado.

SAVEPOINT — um ponto seguro no meio



Resultado: Ana 850 · Bruno 600 · Carla 250 · Davi 50 — total R\$ 1.750,00

SAVEPOINT na prática — um lote de pagamentos

```
BEGIN;
UPDATE conta SET saldo = saldo - 100 WHERE id = 1;    -- Ana -> Bruno
UPDATE conta SET saldo = saldo + 100 WHERE id = 2;
SAVEPOINT pagou_bruno;                                -- ponto seguro

UPDATE conta SET saldo = saldo + 2000 WHERE id = 4;    -- Ana -> Davi
UPDATE conta SET saldo = saldo - 2000 WHERE id = 1;    -- ERRO: saldo -1100
ROLLBACK TO SAVEPOINT pagou_bruno;                    -- desfaz só este

UPDATE conta SET saldo = saldo - 50 WHERE id = 1;      -- Ana -> Carla
UPDATE conta SET saldo = saldo + 50 WHERE id = 3;
COMMIT;
```

Resultado: Ana 850.00 , Bruno 600.00 , Carla 250.00 , Davi 50.00 —
total 1750.00 . Dois pagamentos confirmados, um recusado.

RELEASE e o ROLLBACK sem nome

- `RELEASE SAVEPOINT nome` **descarta o marcador**
 - não grava nada em disco — quem grava é só o `COMMIT`
 - depois dele, `ROLLBACK TO SAVEPOINT nome` dá erro:
`savepoint "nome" does not exist`
- `ROLLBACK` sem nome desfaz **a transação inteira**, com todos os savepoints

Comando	Até onde volta	A transação continua?
<code>ROLLBACK TO SAVEPOINT s</code>	até o <code>SAVEPOINT s</code>	sim
<code>ROLLBACK</code>	até o <code>BEGIN</code>	não

Três cuidados com transações

1. **Transação aberta segura linhas.** Quem tenta alterar a mesma linha **espera** até você confirmar ou desfazer. Não deixe transação aberta esperando o usuário.
2. **Confira a resposta do COMMIT** . Numa transação abortada ele responde **ROLLBACK** .
3. **Saiba em que modo está a sua ferramenta.** No pgAdmin, com *Auto commit?* desligado, cada alteração fica **pendente** até você clicar em **Commit** ou em **Rollback**.

Demonstração — Atividade 1 do laboratório

No ambiente da aula (o mesmo que você terá em casa):

```
cd ~/lab07
podman compose up -d
podman exec -i bd2-aula07-pg psql -U aluno -d banco < preparar.sql
podman exec -it bd2-aula07-pg psql -U aluno -d banco
```

Vamos ver ao vivo:

- a transferência com `BEGIN` / `COMMIT`
- o erro de `CHECK` no meio da transação
- o `COMMIT` respondendo `ROLLBACK`

Resumo da Parte 2

- **Autocommit:** sem `BEGIN`, cada comando se confirma sozinho
- `BEGIN ... COMMIT`: confirma tudo; `ROLLBACK`: desfaz tudo
- **Erro no meio:** a transação fica **abortada** e o `COMMIT` vira `ROLLBACK`
- `SAVEPOINT`: desfaz só um pedaço e deixa a transação continuar
- `RELEASE`: só descarta o marcador

Parte 3

Transações distribuídas e 2PC

Lembra da Aula 04? Dados fragmentados

- **Fragmentação horizontal:** as linhas de uma tabela divididas entre servidores
- A Cooperativa Cerrado cresceu: cada agência tem o **seu servidor**

Servidor	Contas
agência de Palmas	Ana, Bruno
agência de Gurupi	Carla, Davi

A transferência **Ana** → **Davi** agora mexe em **dois SGBDs diferentes**.

Por que o **COMMIT** local não basta

```
-- no servidor de Palmas
BEGIN; UPDATE conta SET saldo = saldo - 300 WHERE id = 1; COMMIT;    -- ok

-- no servidor de Gurupi
BEGIN; UPDATE conta SET saldo = saldo + 300 WHERE id = 4; COMMIT;    -- falhou!
```

- Cada servidor garante o ACID **só dentro dele**
- Palmas já confirmou; Gurupi não — **o dinheiro sumiu de novo**
- E não dá para "desconfirmar" Palmas: o **COMMIT** é **durável**

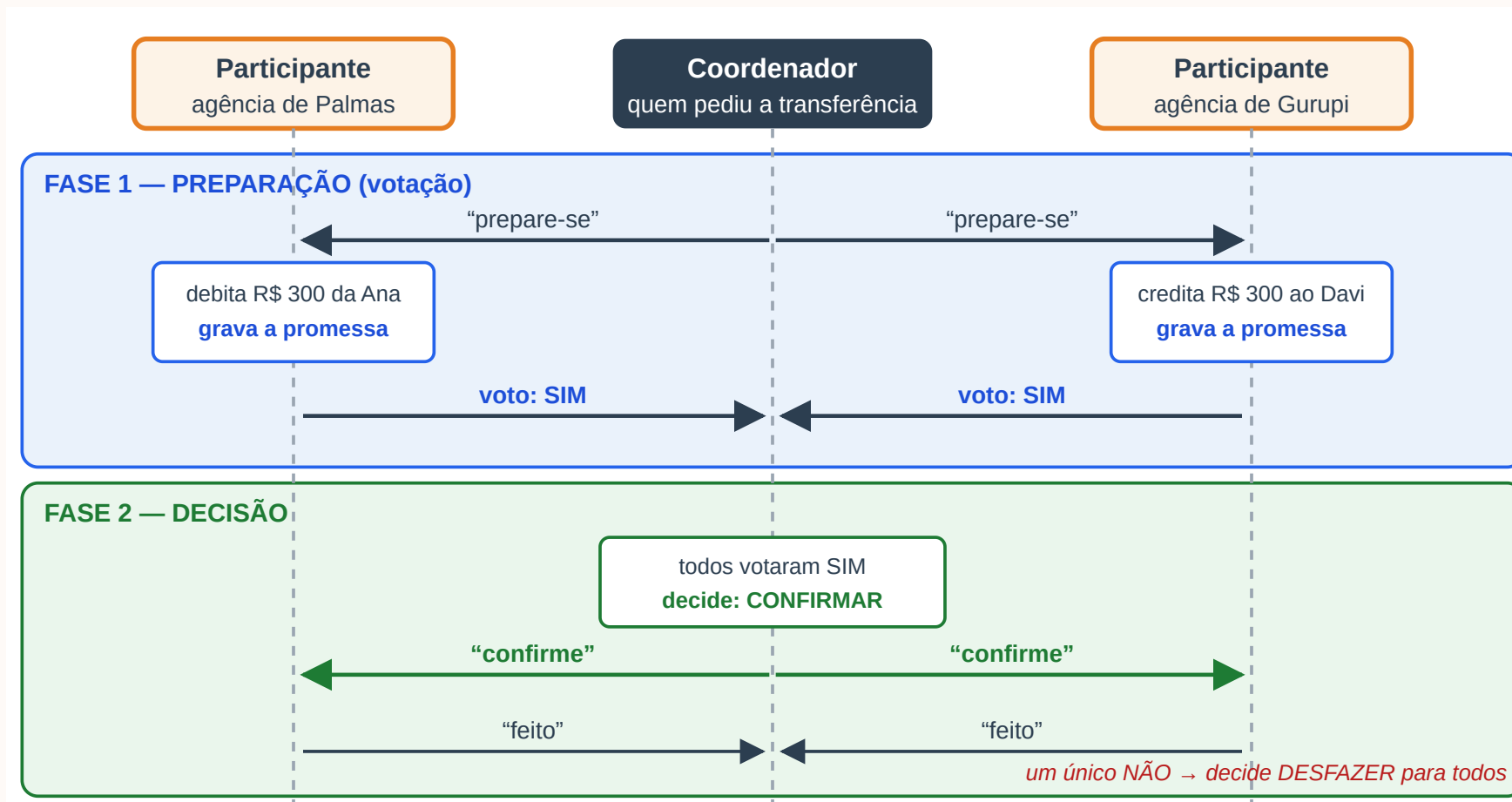
Precisamos que os dois servidores **combinem antes** de confirmar.

Coordenador e participantes

- **Participantes:** os servidores que guardam os dados (Palmas e Gurupi)
- **Coordenador:** quem conduz a transação — normalmente a aplicação ou um **gerenciador de transações**
- A regra de ouro: **ninguém confirma antes de todos poderem confirmar**

A solução clássica é o **protocolo de confirmação em duas fases** — *two-phase commit*, ou **2PC** — descrito por Jim Gray em 1978.

2PC — as duas fases



Fase 1 — Preparação (votação)

- O coordenador pede a cada participante: **"prepare-se"**
- Cada participante faz a sua parte, **sem confirmar**, e:
 - se conseguiu, **grava em disco** tudo o que precisa para confirmar
depois e vota **SIM**
 - se não conseguiu (erro, restrição, falta de espaço), vota **NÃO**
- Votar SIM é uma **promessa**: "se você mandar, eu confirmo — mesmo que eu caia e volte"

Fase 2 — Decisão

Votos recebidos	Decisão do coordenador	Comando para cada participante
todos SIM	confirmar	COMMIT PREPARED
pelo menos um NÃO	desfazer	ROLLBACK PREPARED
alguém não respondeu	desfazer	ROLLBACK PREPARED

- A decisão é **registrada** pelo coordenador e enviada a todos
- Os participantes **obedecem**; não decidem sozinhos
- Resultado: ou **todos** confirmam, ou **todos** desfazem — atomicidade entre servidores

O ponto fraco: o coordenador cai



JANELA DE INCERTEZA

o participante não pode confirmar sozinho, nem desfazer sozinho:
ele prometeu obedecer à decisão do coordenador

a linha da conta continua BLOQUEADA

quem tenta alterá-la espera — mesmo que o servidor reinicie,
porque a promessa foi gravada em disco

O problema do bloqueio

- Depois de votar SIM, o participante **não pode decidir sozinho**: se escolher diferente do coordenador, os servidores ficam inconsistentes
- Enquanto espera, ele **segura as linhas** envolvidas
- A promessa está **em disco**: sobrevive até a um reinício

Por isso o 2PC é um protocolo **bloqueante**: alguém (o coordenador de volta, ou um DBA) precisa **resolver** as transações pendentes.

2PC no PostgreSQL

Comando	Papel no 2PC
<code>BEGIN ... alterações ...</code>	o participante faz a sua parte
<code>PREPARE TRANSACTION 'nome'</code>	fase 1: grava a promessa e vota SIM
<code>COMMIT PREPARED 'nome'</code>	fase 2: confirma
<code>ROLLBACK PREPARED 'nome'</code>	fase 2: desfaz
<code>SELECT * FROM pg_prepared_xacts;</code>	lista as transações preparadas

- Vem **desligado**: é preciso `max_prepared_transactions` maior que zero
- O `compose.yaml` da aula já sobe o servidor com esse parâmetro

2PC no PostgreSQL — Ana → Davi, R\$ 300,00

```
-- Sessão 1: agência de Palmas
BEGIN;
UPDATE conta SET saldo = saldo - 300
WHERE id = 1;
PREPARE TRANSACTION 'transf1-palmas';
```

```
-- Sessão 2: agência de Gurupi
BEGIN;
UPDATE conta SET saldo = saldo + 300
WHERE id = 4;
PREPARE TRANSACTION 'transf1-gurupi';
```

```
SELECT gid, database FROM pg_prepared_xacts ORDER BY gid;
--      gid      |      database
--  transf1-gurupi | agencia_gurupi
--  transf1-palmas | agencia_palmas
```

```
COMMIT PREPARED 'transf1-palmas';
-- Ana: 700.00
```

```
COMMIT PREPARED 'transf1-gurupi';
-- Davi: 350.00
```

O voto "não" no PostgreSQL

```
-- Palmas: Bruno só tem R$ 500,00
BEGIN;
UPDATE conta SET saldo = saldo - 900 WHERE id = 2; -- ERROR: check constraint
PREPARE TRANSACTION 'transf2-palmas'; -- responde: ROLLBACK
```

- Uma transação **abortada** não pode ser preparada: o `PREPARE` responde `ROLLBACK` — esse é o voto **NÃO**
- Gurupi pode ter votado SIM; o coordenador então manda `ROLLBACK PREPARED 'transf2-gurupi'`

Cuidados com o 2PC do PostgreSQL

- `PREPARE TRANSACTION` **não é para a aplicação do dia a dia**: é para **gerenciadores de transação** (padrão XA, JTA no Java, etc.)
- Transação preparada **esquecida** segura bloqueios e impede a limpeza de versões antigas — monitore `pg_prepared_xacts`
- A decisão só pode ser tomada **no banco em que a transação foi preparada**:

```
ERROR:  prepared transaction belongs to another database
HINT:   Connect to the database where the transaction was prepared to finish it.
```

Demonstração — Atividade 5 do laboratório

Duas abas de terminal, uma para cada agência:

```
# Sessão 1
podman exec -it bd2-aula07-pg psql -U aluno -d agencia_palmas
# Sessão 2
podman exec -it bd2-aula07-pg psql -U aluno -d agencia_gurupi
```

Vamos ver ao vivo:

- a fase 1: `PREPARE TRANSACTION` nas duas agências
- o saldo que **ainda não mudou** depois do `PREPARE`
- a fase 2: `COMMIT PREPARED` nas duas

E quando se abre mão do ACID?

Teorema CAP (Brewer, 2000): num sistema distribuído, quando a rede se divide (**P**artição), é preciso escolher entre:

- **C**onsistência — todos veem o mesmo dado, ou o sistema não responde
- **A** — disponibilidade (*availability*): o sistema sempre responde, mesmo com dado desatualizado

Escolha	Exemplo
consistência (CP)	banco, cartório, prontuário — o 2PC fica aqui
disponibilidade (AP)	curtidas, carrinho de compras, recomendações

O modelo BASE

Muitos bancos NoSQL escolhem **disponibilidade** e seguem o modelo **BASE**:

- **B**asically **A**vailable — responde sempre, mesmo com falhas parciais
- **S**oft state — o estado pode mudar sozinho enquanto as réplicas conversam
- **E**ventually consistent — os dados **ficam** consistentes, **com o tempo**

Regra prática: onde errar custa dinheiro, saúde ou direito — financeiro, saúde, jurídico —, use **ACID**.

Resumo da Parte 3

- Com os dados em **dois servidores**, cada `COMMIT` local não basta
- **2PC**: fase 1 — **preparar e votar**; fase 2 — **decidir e obedecer**
- Voto SIM é uma **promessa gravada em disco**
- Ponto fraco: se o coordenador cai, o participante fica **bloqueado**
- PostgreSQL: `PREPARE TRANSACTION` , `COMMIT PREPARED` , `ROLLBACK PREPARED` ,
`pg_prepared_xacts`
- CAP e BASE: quando a disponibilidade vale mais que a consistência imediata

Laboratório

**Sete atividades de até 20 minutos, no
iximiuz Labs**

O ambiente do laboratório

- **Turma (uma vez no semestre):**

<https://labs.iximiuz.com/trainings/aulas-bd2-2026-2-51d1c714> → **Enroll**

- **Playground da aula:**

<https://labs.iximiuz.com/playgrounds/BD2-AULA07-ad3d665d> → **Start Playground**

Aba	Para quê
Sessão 1 e Sessão 2	dois terminais na mesma máquina
IDE	ler os arquivos com calma
pgAdmin	a interface web (e-mail <code>aluno@bd2.unitins.br</code> , senha <code>aluno</code>)

O `compose.yaml` da aula

```
services:
  postgres:
    image: docker.io/library/postgres:17
    container_name: bd2-aula07-pg
    command: ["postgres", "-c", "max_prepared_transactions=10"] # liga o 2PC
    environment: { POSTGRES_USER: aluno, POSTGRES_PASSWORD: aluno, POSTGRES_DB: banco }
    volumes:
      - ./preparar.sql:/docker-entrypoint-initdb.d/01-preparar.sql:ro,z
      - pgdata:/var/lib/postgresql/data
  pgadmin:
    image: docker.io/elestio/pgadmin:latest
    container_name: bd2-aula07-pgadmin
    depends_on: { postgres: { condition: service_healthy } }
    ports: ["8181:80"]
    volumes:
      - ./servers.json:/pgadmin4/servers.json:ro,z # servidor já cadastrado
```

(versão resumida — o arquivo completo está em `~/lab07`)

O preparo — igual em todas as atividades

```
cd ~/lab07  
podman compose up -d  
podman exec -i bd2-aula07-pg psql -U aluno -d banco < preparar.sql
```

- Sobe o **PostgreSQL** e o **pgAdmin**
- **Recria** as contas em `banco`, `agencia_palmas` e `agencia_gurupi`
- Encerra sessões esquecidas e transações preparadas pendentes

Pode rodar quantas vezes quiser. Deu tudo errado? **Rode o preparo de novo.**

As sete atividades

#	O que você vai ver	Tempo
1	Tudo ou nada: <code>BEGIN</code> , <code>COMMIT</code> , <code>ROLLBACK</code> e a transação abortada	15 min
2	<code>SAVEPOINT</code> : desfazer só um pedaço do lote	15 min
3	Isolamento: o que a outra sessão enxerga	15 min
4	Durabilidade: derrubar o servidor e ver o WAL trabalhar	15 min
5	2PC: as duas fases e o voto "não"	20 min
6	2PC: o coordenador caiu	15 min
7	A transação pelo pgAdmin	15 min

Independentes: faça na ordem que quiser, uma, duas ou três por dia.

Atividades 3 e 4 — isolamento e durabilidade

Atividade 3 — duas abas no banco `banco` :

- Sessão 1: `BEGIN` e débito de R\$ 300,00 na Ana (sem confirmar)
- Sessão 2 vê Ana com `1000.00` ; tenta alterar a Ana e **fica esperando**
- Sessão 1: `COMMIT` → a Sessão 2 destrava; Ana fica com `701.00`

Atividade 4 — confirmar uma alteração, deixar outra em aberto e:

```
podman kill bd2-aula07-pg      # "puxar o cabo da tomada"
podman start bd2-aula07-pg
podman logs --tail 10 bd2-aula07-pg
```

O confirmado **sobrevive**; o não confirmado **some**.

Atividades 6 e 7 — bloqueio no 2PC e pgAdmin

Atividade 6: as duas agências votam SIM, o servidor cai antes da decisão e:

- `pg_prepared_xacts` mostra as duas transações **depois** da queda
- `SET lock_timeout = '3s'` e um `UPDATE` na conta presa →
`canceling statement due to lock timeout`
- você decide com `COMMIT PREPARED` em cada agência

Atividade 7: no Query Tool do pgAdmin, desligue **Auto commit?**, faça um `UPDATE` e veja que o terminal **não enxerga** a alteração até você clicar em **Commit**.

Se algo der errado

Sintoma	O que fazer
<code>connection refused</code> / <code>starting up</code>	esperar dez segundos e repetir
<code>current transaction is aborted</code>	<code>ROLLBACK;</code> e recomeçar o trecho
comando parado, sem prompt	confirmar ou desfazer a transação da outra aba
aba pgAdmin não abre	esperar trinta segundos e recarregar
deu tudo errado	rodar o preparo de novo

Tem computador? O mesmo `compose.yaml` roda no **Podman Desktop**; os links dos arquivos estão no roteiro do laboratório.

Resumindo a Aula 07

- **Transação:** unidade indivisível — tudo ou nada
- **ACID:** Atomicidade, Consistência, Isolamento, Durabilidade
- **PostgreSQL:** autocommit por padrão; `BEGIN` , `COMMIT` , `ROLLBACK`
- **Erro no meio:** transação abortada; o `COMMIT` vira `ROLLBACK`
- **SAVEPOINT :** volta só um pedaço
- **Dois servidores: 2PC** — preparar e votar; decidir e obedecer
- **2PC no PostgreSQL:** `PREPARE TRANSACTION` , `COMMIT PREPARED` ,
`ROLLBACK PREPARED`

Para casa e próximos passos

- **Laboratório:** as sete atividades no iximiuz Labs (até 20 min cada)
- **Texto-base da Aula 07** no Google Classroom
- **Questionário:** dez questões no Classroom
- **Aula 08 (03/10):** concorrência, MVCC, bloqueios e níveis de isolamento
- **A1 (10/10):** prova com o conteúdo das Aulas 01 a 06

Referências

- ELMASRI, R.; NAVATHE, S. B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.
- GRAY, J.; REUTER, A. **Transaction processing**: concepts and techniques. San Mateo: Morgan Kaufmann, 1992.
- HÄRDER, T.; REUTER, A. Principles of transaction-oriented database recovery.
ACM Computing Surveys, v. 15, n. 4, p. 287-317, 1983.

Referências (continuação)

- POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 17 documentation:**
Transactions; PREPARE TRANSACTION. 2026.
- SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. **Sistema de banco de dados.**
7. ed. Rio de Janeiro: LTC, 2020.

Fim